

Entwicklung einer Observability-Strategie für die Plattform "frag.jetzt" basierend auf dem Konzept der Four Golden Signals.

Exposé zur Bachelorarbeit im Studiengang Informatik

Leon Rausch

14. Januar 2026

1 Einleitung

Moderne webbasierte Informationssysteme werden zunehmend als verteilte, cloud-native Anwendungen realisiert, die aus einer Vielzahl lose gekoppelter Services bestehen. Diese Architekturen ermöglichen eine hohe Skalierbarkeit und schnelle Weiterentwicklung, erhöhen jedoch gleichzeitig die operative Komplexität im Produktionsbetrieb erheblich [1, 2]. Insbesondere Microservices-basierte Systeme sind durch verteilte Fehlerzustände, schwer nachvollziehbare Abhängigkeiten sowie eine hohe Dynamik gekennzeichnet, was klassische Monitoring-Ansätze an ihre Grenzen bringt [3].

Vor diesem Hintergrund gewinnt der Übergang von reinem Monitoring hin zu umfassender Observability zunehmend an Bedeutung. Observability zielt darauf ab, den internen Zustand eines Systems anhand seiner externen Ausgaben – insbesondere Metriken, Logs und Traces – nachvollziehbar zu machen und dadurch Ursachenanalyse, Fehlerdiagnose und proaktives Handeln zu ermöglichen [4, 1]. Zahlreiche aktuelle Studien zeigen, dass eine systematische Observability-Strategie ein zentraler Erfolgsfaktor für den stabilen Betrieb cloud-nativer Anwendungen ist und maßgeblich zur Reduktion von Incident-Reaktionszeiten sowie zur Erhöhung der Systemzuverlässigkeit beiträgt [5, 2].

Ein weithin anerkanntes konzeptionelles Fundament für effektives Monitoring stellen die von Google Site Reliability Engineering (SRE) definierten *Four Golden Signals* dar: Latenz, Traffic, Fehler und Sättigung [6]. Diese vier Signale adressieren zentrale Aspekte der Nutzerwahrnehmung und Systemauslastung und dienen als minimaler, aber wirkungsvoller Satz an Kernmetriken für produktive Systeme. Google SRE betont dabei, dass insbesondere für verteilte Systeme eine Fokussierung auf wenige, aussagekräftige Signale notwendig ist, um ein günstiges Verhältnis zwischen Signal und Rauschen im Alerting zu gewährleisten [6].

Die Open-Source-Anwendung *frag.jetzt* weist bereits eine ausgereifte CI/CD-Pipeline auf, verfügt jedoch bislang über keine konsistente Observability-Strategie für den produktiven Betrieb. Metriken, Logs, Traces

und Alerting sind nicht systematisch aufeinander abgestimmt, wodurch eine ganzheitliche Sicht auf das Laufzeitverhalten der Anwendung fehlt. Dies erschwert insbesondere die frühzeitige Erkennung von Performanceeinbußen, die Ursachenanalyse bei Incidents sowie die Ableitung handlungsrelevanter Informationen für unterschiedliche Stakeholder-Rollen wie DevOps, Entwickler oder Product Owner [1, 5].

Ziel dieser Arbeit ist daher die Entwicklung einer umfassenden Observability-Strategie für *frag.jetzt* auf Basis der Four Golden Signals. Dabei sollen die Golden Signals servicespezifisch für das PWA-Frontend, das Spring-Boot-Backend, die PostgreSQL-Datenbank sowie angebundene KI-Services von *frag.jetzt* konkretisiert und technisch instrumentiert werden. Ergänzend wird eine vergleichende Evaluation gängiger Observability-Stacks – insbesondere Prometheus/Grafana, ELK Stack und Jaeger – durchgeführt, um eine geeignete Tool-Kombination für die Anforderungen von *frag.jetzt* abzuleiten [7, 5].

Ein weiterer Schwerpunkt liegt auf der Konzeption eines intelligenten Alerting-Frameworks, das False Positives reduziert und rollenbasierte, handlungsorientierte Benachrichtigungen ermöglicht. Aktuelle Forschungsarbeiten zeigen, dass die Kombination aus priorisiertem Alerting und datenbasierten Analyseverfahren die Effektivität von Alerting-Systemen erhöht und die Anzahl irrelevanter Alarme reduziert. [8]. Abschließend sollen operative Runbooks entwickelt werden, die auf den definierten Alerts aufbauen und strukturierte Handlungsanweisungen für wiederkehrende Incident-Szenarien bereitstellen.

2 Theoretische Fundierung: Monitoring und Observability in modernen Systemen

Moderne Softwaresysteme sind zunehmend durch verteilte Architekturen, Cloud-native Technologien und eine hohe Änderungsdynamik geprägt. Insbesondere Microservice-basierte Anwendungen mit Container-Orchestrierung und Continuous Deployment erhöhen die Komplexität des Betriebs erheblich [2]. Klassische Monitoring-Ansätze, die primär auf vordefinierte Metriken und statische Schwellwerte setzen, stoßen in

solchen Umgebungen an ihre Grenzen, da sie häufig nur bekannte Fehlerzustände erfassen und wenig Unterstützung bei der Ursachenanalyse bieten [4].

Vor diesem Hintergrund hat sich der Ansatz der *Observability* als zentrales Konzept für den Betrieb moderner Systeme etabliert. Observability zielt darauf ab, den internen Zustand eines Systems anhand externer Signale nachvollziehbar zu machen und dadurch auch unbekannte oder unerwartete Fehlerszenarien analysierbar zu gestalten [9].

2.1 Begriffliche Grundlagen: Monitoring und Observability

Der Begriff Observability stammt ursprünglich aus der System- und Regelungstheorie und beschreibt die Fähigkeit, interne Zustände eines Systems aus dessen Ausgaben abzuleiten. Übertragen auf Softwaresysteme bezeichnet Observability die Eigenschaft, Systemverhalten durch geeignete Telemetriedaten nachvollziehbar zu machen [4].

Monitoring und Observability sind dabei eng miteinander verbunden, jedoch nicht gleichzusetzen. Monitoring fokussiert sich auf die Überwachung bekannter Metriken und Ereignisse, um Abweichungen vom erwarteten Normalzustand zu erkennen. Observability erweitert diesen Ansatz, indem sie Kontextinformationen bereitstellt, die eine detaillierte Analyse von Ursachen und Zusammenhängen ermöglichen [1]. In der Literatur wird Observability daher häufig als Weiterentwicklung oder Übermenge des klassischen Monitorings beschrieben.

Zentral für Observability ist die systematische Erfassung unterschiedlicher Telemetriearten. Erst durch die kombinierte Betrachtung verschiedener Signale wird es möglich, komplexe Systemzustände sowie Ursachen von Fehlern zuverlässig zu analysieren [9, 2].

2.2 Zentrale Bausteine der Observability: Metriken, Logs, Traces und Alerting

In der Literatur werden vier zentrale Bausteine der Observability unterschieden: Metriken, Logs, Traces und Alerting. Diese Bausteine ergänzen sich gegenseitig und bilden gemeinsam die Grundlage für eine umfassende Analyse des Laufzeitverhaltens moderner Softwaresysteme [9, 2].

Metriken sind numerische Messwerte, die den Zustand oder das Verhalten eines Systems über die Zeit hinweg beschreiben, beispielsweise Antwortzeiten, Fehlerraten oder Ressourcenauslastung. Sie eignen sich besonders für Trendanalysen, Kapazitätsplanung und die Definition von Service-Level-Zielen. Aufgrund ihrer aggregierten Form ermöglichen Metriken eine effiziente Überwachung auch großer Systeme [2].

Logs bestehen aus ereignisbasierten Textinformationen, die detaillierte Einblicke in den Ablauf einzelner Systemkomponenten liefern. Sie enthalten kontextreiche Informationen, etwa Fehlermeldungen oder Zustandsänderungen, und sind insbesondere für die Ursachenanalyse und das Debugging relevant. Der Nachteil von Logs liegt in ihrem hohen Volumen und der vergleichsweise aufwändigen Auswertung [9].

Traces dienen der Nachverfolgung einzelner Anfragen über mehrere Services hinweg. Sie zeigen, welche Komponenten an der Verarbeitung beteiligt waren und wie viel Zeit in den jeweiligen Verarbeitungsschritten verbraucht wurde. Distributed Tracing ist insbesondere in Microservice-Architekturen ein zentrales Mittel zur Identifikation von Latenzengpässen und Abhängigkeitsproblemen [10].

Alerting ergänzt die passive Beobachtung durch aktive Benachrichtigung bei kritischen Zuständen. Alerts werden typischerweise auf Basis von Metriken oder Log-Ereignissen ausgelöst und sollen Betreiber frühzeitig auf Probleme hinweisen. Eine zentrale Herausforderung besteht darin, Alarmregeln so zu gestalten, dass relevante Vorfälle erkannt werden, ohne eine hohe Anzahl an Fehlalarmen zu erzeugen [8].

Erst das Zusammenspiel dieser vier Bausteine ermöglicht eine umfassende Observability. Während Metriken einen schnellen Überblick liefern, stellen Logs und Traces den notwendigen Detailgrad für die Ursachenanalyse bereit. Alerting sorgt schließlich dafür, dass relevante Abweichungen zeitnah erkannt und adressiert werden können.

2.3 Die Four Golden Signals nach Google SRE

Ein etabliertes Referenzmodell zur Strukturierung von Monitoring- und Observability-Daten sind die sogenannten *Four Golden Signals*, die im Rahmen des Site Reliability Engineering (SRE) von Google definiert wurden [6]. Dieses Modell identifiziert vier zentrale Signalbereiche, die den Gesundheitszustand eines Dienstes aus Nutzersicht abbilden:

- **Latency:** Die Zeitspanne zur Bearbeitung einer Anfrage. Hohe oder stark schwankende Latenzen weisen auf Performanceprobleme hin.
- **Traffic:** Das Anfrage- oder Datenvolumen, das auf einen Dienst einwirkt, beispielsweise gemessen in Requests pro Sekunde.
- **Error:** Die Rate fehlgeschlagener Anfragen, etwa durch Serverfehler oder funktionale Fehlzustände.
- **Saturation:** Der Auslastungsgrad kritischer Ressourcen wie CPU, Speicher oder Warteschlangen.

Google empfiehlt, diese vier Signale als primäre Beobachtungsgrößen zu verwenden, da sie eng mit der wahrgenommenen Servicequalität korrelieren und ein gezieltes Alerting ermöglichen [6]. Im Gegensatz zu rein ressourcenorientierten Metriken helfen die Golden

Signals, Monitoring auf nutzerrelevante Symptome zu fokussieren und unnötige Alarmierungen zu vermeiden.

2.4 Die Plattform *frag.jetzt* und ihre Systemarchitektur

Die vorliegende Arbeit bezieht sich exemplarisch auf die Q&A-Plattform *frag.jetzt*, eine Cloud-native Webanwendung mit serviceorientierter Architektur. Die Plattform wurde an der Technischen Hochschule Mittelhessen entwickelt und dient der Durchführung interaktiver Fragerunden. Neben klassischer Frage-Antwort-Funktionalität integriert *frag.jetzt* auch KI-gestützte Assistenzfunktionen zur automatisierten Beantwortung von Nutzeranfragen.

Zur Einordnung der späteren Observability-Strategie wird im Folgenden ein Überblick über die zentralen technischen Komponenten der Plattform gegeben:

- **Angular Frontend (SPA/PWA):** Eine Single-Page-Webanwendung, über die Nutzer mit dem System interagieren. Das Frontend ist als Progressive Web App umgesetzt und kommuniziert überwiegend über HTTPS-APIs sowie WebSockets mit dem Backend.
- **WebSocket-Gateway (NGINX):** Ein Gateway-Server zur Vermittlung der bidirektionalen Echtzeitkommunikation zwischen Frontend und Backend, insbesondere für das asynchrone Pushen von Antworten und Statusupdates.
- **Spring Boot Backend:** Der zentrale, Java-basierte Anwendungsserver, der die Kernlogik der Plattform implementiert. Er verarbeitet Nutzeranfragen, koordiniert interne Dienste und stellt REST- sowie WebSocket-Schnittstellen bereit.
- **Keycloak Identity Service:** Ein externer Dienst zur Authentifizierung und Autorisierung von Nutzern. Er übernimmt das Identitätsmanagement, führt jedoch zusätzliche Abhängigkeiten und potenzielle Latenzpunkte ein.
- **PostgreSQL-Datenbank:** Eine relationale Datenbank zur Persistierung des Anwendungszustands, beispielsweise für Benutzerkonten, Fragen und Antworten. Die Datenbank stellt einen zentralen Bestandteil der Plattform dar und kann bei hoher Last zu einem kritischen Engpass werden.
- **KI-Backend (Langchain & FastAPI):** Ein eigenständiger Microservice, der über Python (FastAPI) und die Langchain-Bibliothek Anfragen an externe Large-Language-Modelle weiterleitet, um KI-generierte Antworten zu erzeugen. Die Abhängigkeit von externen KI-APIs bringt variable Antwortzeiten und zusätzliche Fehlerquellen mit sich.
- **RabbitMQ Message Broker:** Ein Message-Broker zur asynchronen Entkopplung von Backend- und KI-Komponenten. RabbitMQ ermöglicht das Puffern und Weiterleiten von Antwortnachrichten

und unterstützt so eine entkoppelte und skalierbare Verarbeitung.

Diese verteilten Komponenten bilden gemeinsam den durchgängigen Anfragepfad der Plattform. Eine Nutzerfrage durchläuft typischerweise mehrere Dienste, bevor eine Antwort an das Frontend zurückgegeben wird. Dadurch entstehen zahlreiche Schnittstellen und potenzielle Fehlerquellen. Eine Observability-Strategie für *frag.jetzt* muss daher sicherstellen, dass entlang dieses gesamten Pfades geeignete Metriken, Logs und Traces erfasst werden, um Performanceprobleme und Ausfälle nachvollziehbar analysieren zu können [1].

2.5 Werkzeuge für Monitoring und Observability

Zur Umsetzung von Monitoring- und Observability-Konzepten steht eine Vielzahl spezialisierter Open-Source-Werkzeuge zur Verfügung, die in modernen Cloud- und Microservice-Architekturen etabliert sind. Diese Werkzeuge adressieren unterschiedliche Aspekte der Telemetrieerfassung und -auswertung und werden häufig in Kombination eingesetzt [7].

- **Prometheus:** Ein Open-Source-System zur Erfassung und Speicherung zeitbasierter Metriken. Prometheus arbeitet mit einem Pull-basierten Ansatz, bei dem Metrik-Endpunkte regelmäßig abgefragt werden. Die integrierte Abfragesprache PromQL erlaubt detaillierte Analysen von Zeitreihendaten und bildet die Grundlage für regelbasierte Alerting-Mechanismen. Prometheus ist als CNCF-Projekt weit verbreitet und insbesondere für dynamische Cloud-Umgebungen geeignet [7].
- **Grafana:** Eine Plattform zur Visualisierung und Analyse von Observability-Daten aus unterschiedlichen Datenquellen. Grafana dient häufig als zentrales Dashboard-Werkzeug, in dem Metriken, Logs und Traces gemeinsam dargestellt und korreliert werden können. Dadurch unterstützt Grafana die visuelle Analyse von Systemzuständen und Trends [5].
- **Elastic Stack (ELK):** Der Elastic Stack kombiniert Elasticsearch, Logstash und Kibana und wird vor allem für die zentrale Aggregation und Analyse von Logdaten eingesetzt. Die Stärke des Stacks liegt in der skalierbaren Speicherung unstrukturierter Daten sowie in leistungsfähigen Such- und Analysefunktionen. Neben Logs unterstützt Elastic auch Metriken und Tracing, erfordert jedoch vergleichsweise hohe Systemressourcen [7].
- **Jaeger:** Ein verteiltes Tracing-System zur Nachverfolgung von Anfragen über Service-Grenzen hinweg. Jaeger ermöglicht die Analyse von End-to-End-Latenzen und unterstützt die Identifikation von Performanceengpässen sowie Abhängigkeitsbeziehungen in Microservice-Architekturen [10].
- **OpenTelemetry:** Ein herstellerneutraler Standard zur Instrumentierung von Anwendungen.

OpenTelemetry stellt ein einheitliches API und SDK zur Erfassung von Metriken, Logs und Traces bereit und erlaubt die Weiterleitung dieser Daten an unterschiedliche Backends. Dadurch wird die Telemetrieerfassung von der Wahl konkreter Observability-Werkzeuge entkoppelt [10].

Die Auswahl geeigneter Werkzeuge erfolgt typischerweise anhand mehrerer Bewertungskriterien, darunter die unterstützten Telemetriearten, Integrationsfähigkeit, Skalierbarkeit, Analyse- und Visualisierungsfunktionen sowie der Reifegrad der jeweiligen Lösung [5]. Diese Kriterien bilden die Grundlage für eine spätere vergleichende Analyse im Rahmen der Methodik.

3 Methodik / Forschungsdesign

Die Arbeit folgt einem artefaktorientierten Forschungsdesign mit konzeptionellen und prototypischen Anteilen. Ziel ist die Entwicklung einer nachvollziehbaren Observability-Strategie für *frag.jetzt* auf Basis der Four Golden Signals sowie die Ableitung und Implementierung zentraler Artefakte (Observability-Plattform, Dashboards, Alerting-Ansatz, Runbooks). Die Methodik gliedert sich in sechs aufeinander aufbauende Arbeitsschritte, die sich direkt aus der Aufgabenstellung und den Forschungsfragen ableiten.

3.1 Schritt 1: Service-spezifische Konkretisierung der Four Golden Signals

Im ersten Schritt werden die Four Golden Signals (Latency, Traffic, Errors, Saturation) als Referenzmodell herangezogen und für die Zielarchitektur von *frag.jetzt* präzisiert. Dazu werden die Signalebereiche pro Service (PWA-Frontend, Spring Boot Backend, PostgreSQL, KI-Services) konkret definiert und in messbare Indikatoren überführt. Der Fokus liegt auf der Ableitung geeigneter Kennzahlen, die nutzerrelevante Symptome abbilden und als Grundlage für Dashboards und Alerting dienen.

3.2 Schritt 2: Vergleichende Evaluation von Observability-Stack-Kombinationen

Im zweiten Schritt erfolgt eine strukturierte Evaluation gängiger Observability-Stacks, insbesondere Prometheus/Grafana, ELK Stack sowie Jaeger (ggf. ergänzt um OpenTelemetry als Instrumentierungsstandard). Der Vergleich erfolgt anhand nachvollziehbarer Kriterien wie Signalabdeckung (Metriken/Logs/Traces), Integrationsfähigkeit, Skalierbarkeit, Abfrage- und Visualisierungsmöglichkeiten, Betriebsaufwand sowie Reifegrad/Ökosystem.

3.3 Schritt 3: Instrumentierung mit OpenTelemetry

Der dritte Schritt umfasst die geplante Instrumentierung der relevanten Services von *frag.jetzt*. Dabei werden Metriken, Logs und Distributed Tracing so umgesetzt, dass eine konsistente Telemetrie entlang des End-to-End-Anfragepfads möglich wird. OpenTelemetry dient als einheitliche Instrumentierungsschicht zur Erfassung und Weitergabe der Telemetrie an die ausgewählten Backends.

3.4 Schritt 4: Rollenbasierte Dashboards

Im vierten Schritt werden Grafana-Dashboards konzipiert und umgesetzt, die sich an den Informationsbedürfnissen verschiedener Rollen orientieren (DevOps, Entwickler, Product Owner). Ziel ist eine übersichtliche, rollenbezogene Darstellung zentraler Golden-Signal-Indikatoren sowie Drill-Down-Möglichkeiten zur Ursachenanalyse (z. B. von Metrik-Auffälligkeiten zu Traces/Logs).

3.5 Schritt 5: Alerting-Framework mit Fokus auf False-Positive-Reduktion

Im fünften Schritt wird ein Alerting-Framework entworfen, das die Ableitung handlungsorientierter Alerts aus Golden-Signal-Indikatoren ermöglicht. Neben klassischen regelbasierten Alerts werden Konzepte zur Reduktion von False Positives und zur Priorisierung adressiert. Zusätzlich wird die Einbindung datengetriebener Verfahren (z. B. Anomalieerkennung) als Ansatzpunkt für „intelligentes Alerting“ beschrieben.

3.6 Schritt 6: Entwicklung operativer Runbooks

Im sechsten Schritt werden exemplarische Runbooks für wiederkehrende Incident-Szenarien erstellt. Runbooks verknüpfen Alerts mit klaren Diagnose- und Handlungsanweisungen und dienen dazu, Reaktionszeiten zu verkürzen und die operative Qualität der Störungsbearbeitung zu erhöhen.

3.7 Ergebnisartefakte

Aus den beschriebenen Schritten ergeben sich die geplanten Ergebnisse: (1) ein wissenschaftlicher Bericht zur Strategie, Toolauswahl und Alerting-Konzeption, (2) eine lauffähige Observability-Plattform (z. B. Prometheus/Grafana), (3) rollenbasierte Dashboards sowie (4) beispielhafte Runbooks für Incident-Situationen.

4 Geplante Gliederung (konzeptionell)

1. Einleitung

Problemstellung, Motivation, Zielsetzung und Forschungsfragen; Einordnung der Arbeit in den Kon-

text cloud-nativer Systeme und moderner Observability.

2. **Theoretische Fundierung**
Grundlagen zu Monitoring und Observability (Metriken, Logs, Traces, Alerting); Four Golden Signals als Referenzmodell; Überblick relevanter Observability-Tools und Toolklassen.
3. **Untersuchungsgegenstand: Architektur von frag.jetzt**
Beschreibung der Systemarchitektur (PWA-Frontend, Spring Boot Backend, PostgreSQL, KI-Services sowie relevante Schnittstellen/Kommunikationspfade); Ableitung der Observability-Anforderungen je Service.
4. **Methodik / Forschungsdesign**
Vorgehensmodell der Arbeit; Operationalisierung der Golden Signals; Kriterienkatalog für Tool-/Stack-Evaluation; Instrumentierungs- und Evaluationsplan; Validitäts- und Abgrenzungsaspekte.
5. **Evaluation von Observability-Stacks**
Vergleich Prometheus/Grafana, ELK Stack, Jaeger (und OpenTelemetry als Instrumentierungsstandard); Ergebnisdarstellung anhand definierter Kriterien; begründete Auswahl einer geeigneten Stack-Kombination.
6. **Konzeption der Observability-Strategie für frag.jetzt**
Service-spezifische Golden-Signal-Definitionen; Messkonzept (Metriken/Logs/Traces) entlang des Anfragepfads; Telemetrie- und Korrelationserfordernisse.
7. **Umsetzung: Instrumentierung und Observability-Plattform**
Umsetzung der Telemetrie-Erfassung; Konfiguration der Plattformkomponenten; Nachweis der End-to-End-Sicht (Metrik-Trace-Log-Korrelation).
8. **Dashboards und Alerting-Konzept**
Rollenbasierte Dashboards; Alerting-Regeln nach Golden Signals; Ansatz zur False-Positive-Reduktion und Priorisierung; Einbindung von Anomalieerkennung (konzeptionell/prototypisch).
9. **Runbooks und Incident-Playbooks**
Erstellung exemplarischer Runbooks; Verknüpfung von Alerts mit Diagnose- und Handlungsschritten; typische Incident-Szenarien.
10. **Diskussion**
Einordnung der Ergebnisse, Grenzen der Umsetzung, Trade-offs (z. B. Aufwand vs. Nutzen, Datenvolumen/Kardinalität, Abhängigkeit externer KI-Services).
11. **Fazit und Ausblick**
Zusammenfassung der zentralen Ergebnisse; Ausblick auf Weiterentwicklung (z. B. Reifegradsteigerung, SLO-basierte Steuerung, weitere Automatisierung).

Abkürzungsverzeichnis

API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
CPU	Central Processing Unit
ELK	Elasticsearch, Logstash, Kibana
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
KI	Künstliche Intelligenz
NLP	Natural Language Processing
OTel	OpenTelemetry
PWA	Progressive Web App
Q&A	Question and Answer
REST	Representational State Transfer
SLA	Service Level Agreement
SLI	Service Level Indicator
SLO	Service Level Objective
SPA	Single-Page Application
SRE	Site Reliability Engineering
UI	User Interface

Quellen und weiterführende Literatur

- [1] U. Faseeha u. a. „Observability in Microservices: An In-Depth Exploration of Frameworks, Challenges, and Deployment Paradigms“. In: *IEEE Access* 13 (2025), S. 72011–72035. DOI: 10.1109/ACCESS.2025.3562125.
- [2] Kowshik Sakinala. „Monitoring and Observability for Cloud-Native Applications“. In: *Journal of Computer Science and Technology Studies* 7.8 (2025), S. 101–115. URL: <https://al-kindipublishers.org/index.php/jcsts/article/view/10459>.
- [3] S. Fischer u. a. „An Overview of Microservice-Based Systems Used for Evaluation in Testing and Monitoring: A Systematic Mapping Study“. In: (2024). DOI: 10.1145/3644032.3644445.
- [4] Cindy Sridharan. *Distributed Systems Observability*. O'Reilly Media, 2018. URL: <https://unlimited.humio.com/rs/756-LMY-106/images/Distributed-Systems-Observability-eBook.pdf>.
- [5] Luca Giamattei u. a. „Monitoring tools for DevOps and microservices: A systematic grey literature review“. In: *Journal of Systems and Software* 208 (2024), S. 111906. DOI: 10.1016/j.jss.2023.111906.

-
- [6] Rob Ewaschuk. *Monitoring Distributed Systems*. <https://sre.google/sre-book/monitoring-distributed-systems/>. Kapitel 6 in *Site Reliability Engineering*, herausgegeben von Betssy Beyer, Google. 2017.
 - [7] A. Banerjee und R. Singh. „A Comparative Analysis of System Monitoring Architecture: Evaluating Prometheus, ELK Stack, and Custom Dashboards for Performance and Scalability“. In: *International Journal of Computer Applications* (2024). DOI: 10.21275/SR25917095133.
 - [8] Fatemeh Jalalvand u. a. „Alert prioritisation in security operations centres: A systematic survey on criteria and methods“. In: *ACM Computing Surveys* 57.2 (2024), S. 1–36. DOI: 10.1145/3695462.
 - [9] Olumese Anthony Abieba u. a. „Advanced Observability and Monitoring Practices for Enhancing Production Environment Reliability“. In: (2025). URL: <https://worldscientificnews.com/wp-content/uploads/2025/06/WSN-204-2025-270-296.pdf>.
 - [10] Carlos Albuquerque und Filipe F Correia. „Tracing and Metrics Design Patterns for Monitoring Cloud-native Applications“. In: *arXiv preprint arXiv:2510.02991* (2025). URL: <https://arxiv.org/abs/2510.02991>.

Hinweis zur KI-Nutzung: Bei der Erstellung dieses Exposés wurden KI-Tools (GitHub Copilot, ChatGPT, Consensus, Elicit) zur Unterstützung bei Recherche und Formulierung eingesetzt. Die inhaltliche Verantwortung liegt beim Autor.